



3



12



1800 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

لغة الحاسوب

د. خولة العلي



RB Informatics; 06/05/2026

بنيان الحاسوب 1

السلام عليكم ورحمة الله وبركاته

مقدمة:

كل حاسوب له لغة خاصة التي يفهمها المعالج، وتتكون هذه اللغة من مجموعة تعليمات التي تكتب لتنفيذ عمليات مختلفة داخل الحاسوب
لاحظنا في اللغات عالية المستوى ان كلمات اللغة تكون اختصارات من كلمات اجنبية ومعاملاتها فهي متغيرات او ثوابت معرفة ضمن البرنامج .

هل المعاملات بالنسبة للغة المعالج هي متغيرات أيضا؟

طبعاً لا، المعاملات في لغة المعالج هي سجلات وليس متغيرات او ثوابت مثل لغات عالية المستوى

- التعليمات Instruction: هي كلمات لغة الحاسوب
- Instruction set: وهي مجموعة أوامر تفهمها بنية الحاسوب

ما هو بنيان الحاسوب Architecture

هو عبارة عن مجموعة الدارات كل دائرة تستقبل مجموعة تعليمات .

للحواسيب لغات متشابهة جداً , نستطيع تشبيهها بلهجات مناطقية أكثر من انها لغات مختلفة كلياً , ويكون من السهل تعلم لغات أخرى بمجرد ان نتعلم لغة واحدة منها
كما ان مصممي الحاسوب لديهم هدف مشترك وهو إيجاد لغة تسهل بناء الـ hardware و compiler بحيث تحقق أقصى قدر من الأداء واقل تكلفة واستهلاك للطاقة.



Mistakes are the portals of discovery

ISA

هي اختصار لـ Instruction set architecture يمكننا القول انها مرتبطة بكل ما يخص التنفيذ في الـ cpu .

وعند تصميم أي ISA يجب تحديد العناصر الأساسية التالية:

1. السجلات registers: هي ذواكر صغيرة سريعة موجودة في وحدة المعالجة المركزية

وظيفتها: حمل التعليمات المنفذة حاليا وحمل معاملاتها ولكي يتعامل معها ISA يجب ان يتم تصنيفها و تسميتها والمسؤول عن ذلك هو المعالج

2. أنماط نقل البيانات: التعليمات التي تنظم حركة البيانات بين السجلات والذاكرة وأجهزة الدخل والخرج

3. كفاءة التنفيذ (الكافية): يجب ان تحتوي الـ ISA على تعليمات كافية لترجمة أي برنامج بكفاءة الى لغة الالة.

4. تنسيق التعليمات: التمثيل الثنائي (binary) المستخدم, وهل التعليمات ثابتة الطول (fixed _ length) او متغيرة الطول (variable _ length)

↪ أنواع الـ ISA:

تنقسم معماريات الحاسوب الى نوعين رئيسيين بناء على كيفية تصميم تعليماتها :

1. حاسوب مجموع التعليمات المعقدة CISC وهي اختصار لـ complex instruction set computer :

ميزاته:

- لديه العديد من التعليمات (عدة مئات)
- التعليمات فيه معقدة أي الأمر الواحد يمكن ان يستغرق تنفيذه عدة دورات
- مثال : معالجات intel x86 (موجودة في معظم أجهزة اللابتوب والكمبيوتر المكتبي)
- Note : هذا المعالج حله تقنية او برمجية

2. حاسوب مجموع التعليمات المختزلة RISC وهو اختصار لـ Reduce instruction set computer :

ميزاته:

- تحتوي على مجموعة بسيطة وصغيرة من التعليمات (عادة 32 تعليمة)
- التعليمات بسيطة أي كل امر مصمم ليقوم بمهمة واحدة بسيطة .
- كل تعليمة تنفذ في دورة ساعة واحدة تقريبا .
- الاستخدام الفعال لتقنية التوارد pipelining
- (أي يسمح للمعالج بالعمل على عدة التعليمات في نفس الوقت)
- مثال: 1. ARM الموجود في هواتف الذكية .

2. MIPS

لو كان لدينا 10 صناديق ونريد ان نقوم بنقلهم من طرف طريق الى الطرف الاخر
CISC: فينا نشبهها شاحنة ضخمة وقوية قادرة تحمل ال 10 صناديق سوا لكنها ستكون بطيئة بالحركة وتستهلك وقت طويل.
RISC : فينا نشبهها ل دراجة قادرة تحمل صندوق واحد فقط لكنها ستكون سريعة جدا بالتالي رح تروح وترجع اكثر من مرة.

مقاييس الأداء:

Execution time (زمن التنفيذ):

هو الوقت الإجمالي الذي يحتاجه الحاسوب لاتمام مهمة محددة بالكامل (منذ وقت الطلب الى وقت التنفيذ) ويتضمن:

- Disk accesses: يمثل الوقت الضائع في قراءة البيانات من الهاردسك
- Memory accesses: يمثل وقت جلب البيانات من RAM
- I/O activates: مثل التعامل مع الشاشة و وحدة التحكم
- Operating system overhead عبء نظام التشغيل : هو الوقت الذي يستهلكه النظام لإدارة مهمة.
- Cup execution time: هو الوقت الفعلي الذي قضاه المعالج في الحساب
- وغيرها..

Clock Cycles

معظم الحواسيب مبنية باستخدام ساعة داخلية هي التي تحدد متى تقع الاحداث داخل الـ Hardware ولذلك نظام الحاسوب هو نظام أحداث متقطع.

مهمتها:

تقسيم الأوقات في الحاسوب الى فترات زمنية منفصلة تسمى دورات الساعة clock cycles ولها مسميات أخرى مثل: clock periods cycles , clock , clock ticks , ticks.



للفهم

المعالج يعمل باستخدام clock تقسم الزمن الى دورات تسمى clock cycles , واذا كان التردد (سرعة المعالج) 3GHz فهذا يعني 3 مليارات دورة في الثانية
- اي أن سرعة تنفيذ البرنامج تعتمد على عدد الـ cycles اللازمة للتنفيذ وبالتالي كلما قل عدد الـ cycles وزاد التردد كان التنفيذ اسرع.

ويوجد مسميات أخرى لزمان التنفيذ :

Wall – clock time

Elapsed time

Response time

العلاقة بين تنظيم الحاسوب وبنية الحاسوب :

نعلم أن:

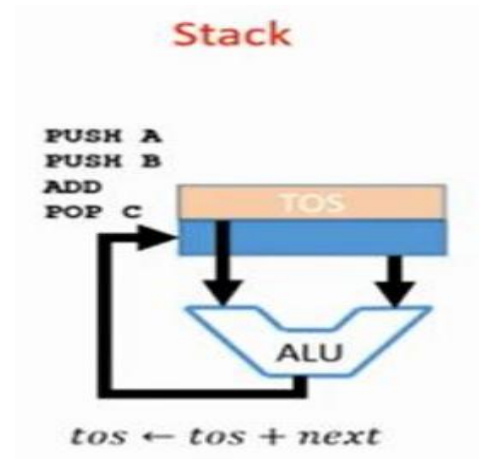
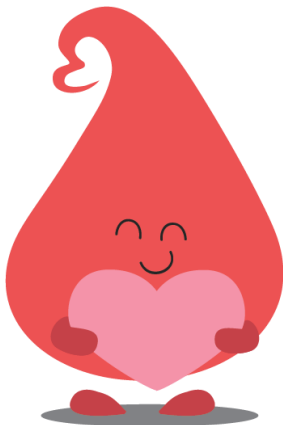
تنظيم الحاسوب: أي كيف يفعل.

بنية الحاسوب: أي ماذا يفعل

:ISA Types

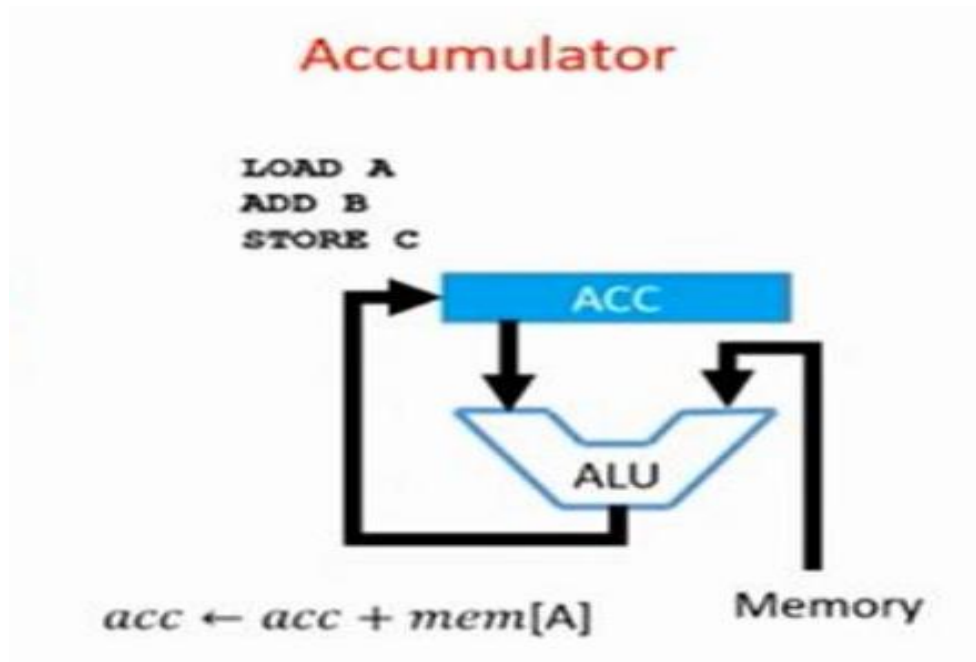
تعتمد المعالجات في تنفيذ العمليات على ثلاثة أنماط رئيسية لتنظيم العمل (مثل: لدينا تعليمة الجمع التالية $c=a+b$).

Stack Architecture



تجمع القيمة الموجودة في القمة Tos مع القيمة التالية ويوضع الناتج في Tos
تعتمد العمليات على دفع البيانات وسحبها من قمة المكس

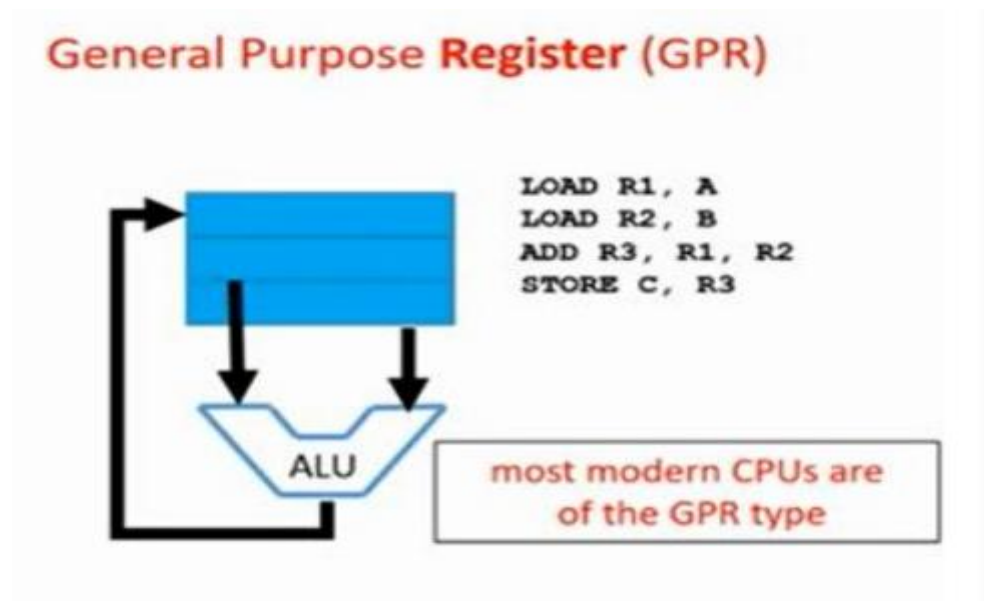
Accumulator Architecture ↙



أي عملية تتم يجب ان يكون ل Acc احد أطرافها بحيث في المثال يحمل القيمة A وتضيف اليه B وتخزن القيمة النهائية في Acc.

أي: Acc يحمل قيمة A وقيمة النتيجة لاحقا
تعتمد العمليات على سجل واحد رئيسي يسمى المراكم (acc)

General purpose Register ↙



هذا هو النظام المستخدم في المعالجات الحديثة



كيف يعمل؟ المعالج لديه مجموعة صناديق (تدعى سجلات) مرقمة $R1, R2, \dots$ يمكن استخدام أي صندوق

لاي غرض في مثالنا السابق

1. نضع A في السجل R1

2. نضع B في السجل R2

3. نجمع R1, R2 ونضع النتيجة في R3

⚡ ملاحظات:

1- تختلف هذه الأنماط في مكان وجود المعاملات (operands) ففي معمارية المكسد (stack) تكون في الذاكرة فقط بينما في معمارية سجلات الأغراض العامة (GPR) تكون في السجلات وفي معمارية المراكم (Accumulator) قد تكون في كليهما (الذاكرة والسجلات).

2- ماهي أنواع العمليات التي يمكن ان تنفذها مجموعة تعليمات المعالج ISA ؟ عمليات حسابية ومنطقية.

⚡ للاطلاع

MIPS Reference Data				①	
CORE INSTRUCTION SET				OPCODE / FUNCT (Hex)	
NAME, MNEMONIC	FOR-MAT	OPERATION (in Verilog)			
Add	add	R	$R[rd] = R[rs] + R[rt]$	(1)	0 / 20 _{hex}
Add Immediate	addi	I	$R[rt] = R[rs] + \text{SignExtImm}$	(1,2)	8 _{hex}
Add Imm. Unsigned	addiu	I	$R[rt] = R[rs] + \text{SignExtImm}$	(2)	9 _{hex}
Add Unsigned	addu	R	$R[rd] = R[rs] + R[rt]$		0 / 21 _{hex}
And	and	R	$R[rd] = R[rs] \& R[rt]$		0 / 24 _{hex}
And Immediate	andi	I	$R[rt] = R[rs] \& \text{ZeroExtImm}$	(3)	0 _{hex}
Branch On Equal	beq	I	$\text{if}(R[rs] == R[rt])$ $PC = PC + 4 + \text{BranchAddr}$	(4)	4 _{hex}
Branch On Not Equal	bne	I	$\text{if}(R[rs] != R[rt])$ $PC = PC + 4 + \text{BranchAddr}$	(4)	5 _{hex}
Jump	j	J	$PC = \text{JumpAddr}$	(5)	2 _{hex}
Jump And Link	jal	J	$R[31] = PC + 8; PC = \text{JumpAddr}$	(5)	3 _{hex}
Jump Register	jr	R	$PC = R[rs]$		0 / 08 _{hex}
Load Byte Unsigned	lbu	I	$R[rt] = [24'b0, M[R[rs] + \text{SignExtImm}](7:0)]$	(2)	24 _{hex}
Load Halfword Unsigned	lhu	I	$R[rt] = [16'b0, M[R[rs] + \text{SignExtImm}](15:0)]$	(2)	25 _{hex}
Load Linked	ll	I	$R[rt] = M[R[rs] + \text{SignExtImm}]$	(2,7)	30 _{hex}
Load Upper Imm.	lui	I	$R[rt] = [\text{imm}, 16'b0]$		1 _{hex}
Load Word	lw	I	$R[rt] = M[R[rs] + \text{SignExtImm}]$	(2)	23 _{hex}

Instruction Type	Opcode	Representation	Description
Data transfer	00001010	LOAD MQ	Transfer contents of register MQ to the accumulator AC
	00001001	LOAD MQ,M(X)	Transfer contents of memory location X to MQ
	00100001	STOR M(X)	Transfer contents of accumulator to memory location X
	00000001	LOAD M(X)	Transfer M(X) to the accumulator
	00000010	LOAD -M(X)	Transfer -M(X) to the accumulator
	00000011	LOAD M(X)	Transfer absolute value of M(X) to the accumulator
Unconditional branch	00000100	LOAD - M(X)	Transfer - M(X) to the accumulator
	00001101	JUMP M(X,0:19)	Take next instruction from left half of M(X)
	00001110	JUMP M(X,20:39)	Take next instruction from right half of M(X)
Conditional branch	00001111	JUMP+ M(X,0:19)	If number in the accumulator is nonnegative, take next instruction from left half of M(X)
	00010000	JUMP+ M(X,20:39)	If number in the accumulator is nonnegative, take next instruction from right half of M(X)
Arithmetic	00000101	ADD M(X)	Add M(X) to AC; put the result in AC
	00000111	ADD M(X)	Add M(X) to AC; put the result in AC
	00000110	SUB M(X)	Subtract M(X) from AC; put the result in AC
	00001000	SUB M(X)	Subtract M(X) from AC; put the remainder in AC
	00001011	MUL M(X)	Multiply M(X) by MQ; put most significant bits of result in AC, put least significant bits in MQ
	00001100	DIV M(X)	Divide AC by M(X); put the quotient in MQ and the remainder in AC
	00010100	LSH	Multiply accumulator by 2; i.e., shift left one bit position
	00010101	RSH	Divide accumulator by 2; i.e., shift right one position
Address modify	00010010	STOR M(X,8:19)	Replace left address field at M(X) by 12 rightmost bits of AC
	00010011	STOR M(X,28:39)	Replace right address field at M(X) by 12 rightmost bits of AC

Table 2.1 The IAS Instruction Set



TOP – down approach التدرج من الأعلى للأسفل

لماذا سمي هكذا؟

تبدأ البرمجة بلغة عالية المستوى (مثل C) ثم يقوم المترجم (compiler) بتحويلها الى لغة تجميع (Assembly) وأخيرا يقوم المجمع (Assembler) بتحويلها الى لغة الآلة (Machine code) المكونة من اصفار وواحدات وهي اللغة التي تفهمها الدوائر المنطقية في المعالج. وبتفصيل اكثر لدينا ثلاث مراحل:

1. High-level language (مستوى عالي):

$A+B=C$

هذه الطريقة الي يكتب فيها المبرمج باي لغة برمجة وهي عبارة رياضية واضحة ومفهومة للبشر.

2. Assembly language (مستوى متوسط):

`add $s1,$s2,$s3`

($s3$ مصدر 2, $s2$ مصدر 1, $s1$ الوجهة)

في هذه المرحلة تبدأ أهمية ISA لأنها تقوم بتحويل كل تعليمة الى اصفار وواحدات لكي يفهمها الحاسوب.

3. Machine code (مستوى ادنى):

وهي عبارة عن سلسلة اصفار وواحدات يفهمها الحاسوب وتقوم بتنفيذها.

ولان سنختار MIPS كمثال للغة المعالج..... فهي اختصارا ل

Microprocessor without Interlocked pipelined Stages.

ملاحظة: طور اول معالج MIPS عام 1981 من قبل جون هينسي وطلابه في جامعة ستانفورد وحاليا تملكه شركة Imagination Technologies



أهمية MIPS في سوق الأنظمة المدمجة :

- يحتل حصة كبيرة جدا في الأجهزة التي نستخدمها يوميا مثل :
- Strong Equipment , Cameras , Printes , Network , etc...
- يحتوي على أكثر 65 تعليمة .
- يعتبر MIPS لغة نموذجية لبقية المعالجات فإذا فهمنا كيف يعمل سنفهم بسهولة كيف تعمل معظم المعالجات الأخرى....

أنواع العمليات في MIPS:

1. العمليات الحسابية (Arithmetic): add, sub, addi
2. العمليات المنطقية (logical) : and, or, nor, andi....
3. نقل البيانات (Data Transfer): lw: جلب البيانات من الذاكرة ووضعها في السجل
sw: جلب البيانات من السجل وتخزينها في الذاكرة
4. التفرع الشرطي (conditional Branch)
5. القفز غير المشروط (Unconditional Jump).

للاطلاع

MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add \$s1,\$s2,\$s3	$s1 = s2 + s3$	Three register operands
	subtract	sub \$s1,\$s2,\$s3	$s1 = s2 - s3$	Three register operands
	add immediate	addi \$s1,\$s2,20	$s1 = s2 + 20$	Used to add constants
Data transfer	load word	lw \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$	Word from memory to register
	store word	sw \$s1,20(\$s2)	$\text{Memory}[s2 + 20] = s1$	Word from register to memory
	load half	lh \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$	Halfword memory to register
	load half unsigned	lhu \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$	Halfword memory to register
	store half	sh \$s1,20(\$s2)	$\text{Memory}[s2 + 20] = s1$	Halfword register to memory
	load byte	lb \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$	Byte from memory to register
	load byte unsigned	lbu \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$	Byte from memory to register
	store byte	sb \$s1,20(\$s2)	$\text{Memory}[s2 + 20] = s1$	Byte from register to memory
	load linked word	ll \$s1,20(\$s2)	$s1 = \text{Memory}[s2 + 20]$	Load word as 1st half of atomic swap
	store condition. word	sc \$s1,20(\$s2)	$\text{Memory}[s2 + 20] = s1; s1 = 0 \text{ or } 1$	Store word as 2nd half of atomic swap
Logical	load upper immedi.	lui \$s1,20	$s1 = 20 * 2^{16}$	Loads constant in upper 16 bits
	and	and \$s1,\$s2,\$s3	$s1 = s2 \& s3$	Three reg. operands; bit-by-bit AND
	or	or \$s1,\$s2,\$s3	$s1 = s2 s3$	Three reg. operands; bit-by-bit OR
	nor	nor \$s1,\$s2,\$s3	$s1 = \sim(s2 s3)$	Three reg. operands; bit-by-bit NOR
	and immediate	andi \$s1,\$s2,20	$s1 = s2 \& 20$	Bit-by-bit AND reg with constant
	or immediate	ori \$s1,\$s2,20	$s1 = s2 20$	Bit-by-bit OR reg with constant
	shift left logical	sll \$s1,\$s2,10	$s1 = s2 \ll 10$	Shift left by constant
Conditional branch	shift right logical	srl \$s1,\$s2,10	$s1 = s2 \gg 10$	Shift right by constant
	branch on equal	beq \$s1,\$s2,25	if ($s1 == s2$) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne \$s1,\$s2,25	if ($s1 \neq s2$) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt \$s1,\$s2,\$s3	if ($s2 < s3$) $s1 = 1$; else $s1 = 0$	Compare less than; for brq, bne
	set on less than unsigned	sltu \$s1,\$s2,\$s3	if ($s2 < s3$) $s1 = 1$; else $s1 = 0$	Compare less than unsigned
	set less than immediate	slti \$s1,\$s2,20	if ($s2 < 20$) $s1 = 1$; else $s1 = 0$	Compare less than constant
	set less than immediate unsigned	sltiu \$s1,\$s2,20	if ($s2 < 20$) $s1 = 1$; else $s1 = 0$	Compare less than constant unsigned
Unconditional jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr \$ra	go to \$ra	For switch, procedure return
	jump and link	jal 2500	$ra = PC + 4$; go to 10000	For procedure call

➤ نعلم مسبقا ان MIPS يعتمد على مبدأ RISC ويعتمد على القواعد التالية:

1. التعليمات تنفذ وظائف بسيطة .
2. الحفاظ على انتظام التنسيق كل تعليمة في MIPS هي عبارة عن كلمة واحدة وطولها ثابت (bits32) تحتوي التعليمة على كود التشغيل (OPcode) و معاملات (operands) بشكل منتظم مما سهل قراءتها بسرعة.
3. تقليل الوصول للذاكرة الذاكرة بطيئة لذلك تستخدم السجلات وكلما كان ذلك ممكن ولذلك لا نذهب الى الذاكرة لجلب البيانات او تخزينها الا للضرورة لذلك نستخدم السجلات كوسائط كلما امكن ذلك , وبذلك لا نذهب الى الذاكرة...

➤ أنواع التعليمات في MIPS:

1. نوع (R-type): تستخدم السجلات فقط كمعاملات وهي اختصار ل Register type.
2. نوع (I-type): معاملاتها تكون سجلات وأرقام ثابتة (عناوين-اعداد) وهي اختصار ل Immediate type.
3. نوع القفز (J-type): تستخدم للقفز الى عنوان محدد في الكود (الانتقال الى سطر معين) وهي اختصار ل Jump type.

➤ عمليات ومعاملات MIPS الحسابية :

- ❖ في لغة MIPS أي عملية جمع او طرح يجب ان تحتوي على ثلاثة معاملات .
- 1. مصدران (Two sources): هي الأرقام التي نريد طرحها او جمعها
- 2. وجهة واحدة (One Destination): المكان الذي سنضع فيه النتيجة.

➤ مبادئ التصميم في MIPS : هما مبدأين:

1. البساطة تعزز الانتظام (Simplicity Favors Regularity)
2. الأصغر هو الأسرع (Smaller is Faster)



المبدأ التصميمي الأول

➤ لماذا يجبرنا MIPS دائما على استخدام 3 معاملات ؟

1. لان جعل جميع التعليمات بنفس الشكل (نفس عدد المعاملات) يجعل تصميم Hardware ابسط.
2. الأنظمة يسهل عملية ترميز (encoding) التعليمات وتنفيذها بسرعة اعلى وتكلفة اقل .

➤ كيف نجمع اكثر من رقمين؟

- مثال: $a = b + c + d + e$ بما ان MIPS لا يقبل الا رقمين في كل مرة لذلك يجب ان نكتب 3 اسطر من لغة التجميع Assembly.

- Add a , b , c # $a = b + c$
- Add a , a , d # $a = (b + c) + d$
- Add a , a , e # $a = (b + c + d) + e$

السجلات في MIPS:

- العدد: يوجد 32 سجلا في بنية MIPS
- الوظيفة: تستخدم من اجل تخزين البيانات التي يتم الوصول اليها بشكل متكرر.
- الحجم: كل سجل حجمه 32 بت ويطلق على هذه ال 32 بت اسم (WORD) كلمة.
- الترقيم: ترقيم من 0 — 31
- مثالا: \$31 , \$1 , , \$0

أسماء السجلات في لغة التجميع {Assembler name}:

❖ بدلا من استخدام الأرقام فقط نحن نستخدم أسماء تسهل على المبرمج معرفة وظيفة السجل.

1. السجلات المؤقتة: ترمز \$t0 , \$t1 , , \$t9
هذه تستخدم لتخزين القيم المؤقتة (لا نحتاجها لفترات طويلة) (Temporary values)
وتقابل السجلات من ((8 15)) \$t0 , , \$t7

2. السجلات المحفوظة: ترمز \$s0 , \$s1 , , \$s7
تستخدم لتخزين المتغيرات المهمة (المحفوظة) (saved variables)
وتقابل السجلات من: ((16 23)) \$s0 , , \$s7

ملاحظة : : الأرقام يفهمها ال CPU والاسماء يفهمها ال Assembler.

ولدينا أيضا سجلات ذات وظائف خاصة:

- السجل الصفري: وهو السجل رقم 0 وتكون قيمته دائما صفرا ويستخدم عند الحاجة لإجراء عمليات تتضمن الصفر
- سجل (IR_Instruction Register): السجل المسؤول عن حمل وتخزين التعليمة التي يتم تنفيذها حاليا.
- السجل (pc _ program counter) : السجل الذي يحمل عنوان التعليمة التالية المقرر تنفيذها .

المبدأ التصميمي الثاني : (الأصغر هو الأسرع)

لماذا يوجد 32 سجلا فقط؟

- الرغبة في زيادة السرعة هي السبب الرئيسي وراء اكتفاء MIPS ب 32 سجلا بدلا من مئات او الاف السجلات .

ما تأثير كثرة السجلات؟

1. زيادة زمن دورة الساعة (Clock cycle time - cct) مما يعني جعل المعالج ابطا

للفهم: زيادة عدد السجلات تؤدي الى زيادة المسافة التي يجب ان تقطعها الإشارة الكهربائية داخل المعالج هذه المسافة الزائدة قد تؤدي الى زيادة زمن دورة الساعة.

2. كلما زاد عدد السجلات احتجنا الى عدد اكبر من عدد البتات داخل تعليمة البرمجية لتمثيل عنوان السجل (مما يجعل حجم التعليمات اكبر واكثر تعقيدا).

تطبيق :

لدينا السطر التالي مكتوب بلغة البرمجة C

$$f = (g + h) - (i + j);$$

المعالج لا يعرف المتغيرات هكذا (g, f, h) لذلك يجب ان تخصص لكل متغير سجلا ليحفظ قيمته

f → \$s0

g → \$s1

h → \$s2

i → \$s3

j → \$s4

وبما ان MIPS لا يستطيع تنفيذ العملية كاملة في سطر واحد سنقوم بتقسيمها.

1. نجمع g + h ونخزنها في سجل مؤقت \$t0.

add \$t0, \$s1, \$s2

2. نجمع i + j ونخزنها في سجل مؤقت \$t1

add \$t1, \$s3, \$s4

3. عميلة الطرح: نطرح \$t0 من \$t1 ونخزنها في السجل النهائي \$s0 (تمثل f) الذي يمثل الواجهة.

sub \$s0, \$t0, \$t1

الصيغة المكافئة بالأرقام:

السجلات لها ارقام أيضا يمكن كتابة الكود باستخدام ارقام السجلات مباشرة

السجل \$s0 هو 16

السجل \$s1 هو 17

السجل \$t0 هو 8

فتصبح:

add \$8, \$17, \$16

add \$9, \$19, \$20

sub \$16, \$8, \$9

